

Prova Prática – Engenheiro de Dados

Desafio prático para avaliação na vaga de Engenheiro de Dados. O projeto deve ser capaz de lidar com dados incrementais, manter histórico e disponibilizar informações consolidadas para análise.

Você recebeu um arquivo Excel (`dados_entrada.xlsx`) contendo eventos de mudanças para duas entidades relacionadas: **clientes** e **endereços**. Ambas compartilham o mesmo identificador `id_cliente`, e cada aba do Excel representa uma dessas entidades.

Sua tarefa é construir um pipeline de dados em Python que realize as seguintes etapas:

Padrão de Nomenclatura dos Recursos

Os seguintes padrões serão aplicados automaticamente para sua conta:

- **Bucket S3:** `bkt-dev1-data-avaliacoes`
- **Pasta do usuário:** `{nome}_{sobrenome}/` (todos em minúsculas)

Pastas no S3 (use exatamente estes caminhos):

- Raw - Clientes: `s3://bkt-dev1-data-avaliacoes/{nome}_{sobrenome}/raw/clientes/`
- Raw - Endereços: `s3://bkt-dev1-data-avaliacoes/{nome}_{sobrenome}/raw/enderecos/`
- Stage - Clientes: `s3://bkt-dev1-data-avaliacoes/{nome}_{sobrenome}/stage/clientes/`
- Stage - Endereços: `s3://bkt-dev1-data-avaliacoes/{nome}_{sobrenome}/stage/enderecos/`
- Analytics - Clientes: `s3://bkt-dev1-data-avaliacoes/{nome}_{sobrenome}/analytics/clientes/`
- Athena Results: `s3://bkt-dev1-data-avaliacoes/{nome}_{sobrenome}/athena_results/`

Recursos AWS:

- **Crawler Glue:** `{nome}_{sobrenome}_crawler`
- **Database Glue:** `{nome}_{sobrenome}`
- **Usuário IAM:** `{nome}_{sobrenome}_user`
- **IAM Role para Crawler:** `{nome}_{sobrenome}_glue_crawler_role`

Configuração no Código:

No seu arquivo `.env` ou `config.py`, use as variáveis de ambiente:

S3_BUCKET=bkt-dev1-data-avaliacoes
AWS_REGION=sa-east-1
AWS_ACCESS_KEY_ID=<sua_access_key>
AWS_SECRET_ACCESS_KEY=<sua_secret_key>

Estrutura dos Dados de Entrada

Aba clientes:

Campo	Tipo	Obrigatório	Descrição
id_cliente	int	Sim	Identificador único do cliente
nome	string	Sim	Nome completo do cliente
email	string	Sim	E-mail do cliente (formato válido)
cpf	string	Sim	CPF do cliente (formato: XXX.XXX.XXX-XX)
data_nascimento	date	Sim	Data de nascimento (formato: YYYY-MM-DD)
status	string	Sim	Status do cliente: ativo, inativo ou suspenso
data_evento	datetime	Sim	Timestamp do evento de mudança (YYYY-MM-DD HH:MM:SS)

Aba enderecos:

Campo	Tipo	Obrigatório	Descrição
id_endereco	int	Sim	Identificador único do endereço (PK)
id_cliente	int	Sim	Identificador do cliente (FK)
cep	string	Sim	CEP (formato: XXXXX-XXX)
logradouro	string	Sim	Endereço completo
numero	string	Sim	Número do endereço
complemento	string	Não	Complemento do endereço (opcional)
bairro	string	Sim	Bairro
cidade	string	Sim	Cidade
estado	string	Sim	UF (sigla com 2 letras: SP,

Campo	Tipo	Obrigatório	Descrição
			MG, RJ, etc)
data_evento	datetime	Sim	Timestamp do evento de mudança (YYYY-MM-DD HH:MM:SS)

Nota importante sobre as mudanças de Endereços: O campo `id_endereco` é a **chave primária** para a tabela de endereços. Múltiplos `id_endereco` podem compartilhar o mesmo `id_cliente` (um cliente pode ter múltiplos endereços). Para o merge incremental, use **`id_endereco`** como chave de identificação, não `id_cliente`.

Parte 1 – Ingestão e Armazenamento Raw

1. **Leia o arquivo Excel** (`dados_entrada.xlsx`) com as duas abas:
 - Aba **clientes**: contém eventos de mudanças para clientes
 - Aba **enderecos**: contém eventos de mudanças para endereços
2. **Implemente validações de qualidade de dados conforme as regras definidas acima:**
 - Validação de campos obrigatórios (não nulos e não vazios)
 - Validação de formato de CPF (XXX.XXX.XXX-XX)
 - Validação de formato de CEP (XXXXX-XXX)
 - Validação de e-mail (contém @ e domínio válido)
 - Validação de datas (formato YYYY-MM-DD e valores válidos)
 - Validação de status (apenas: ativo, inativo, suspenso)
 - Validação de integridade referencial: `id_cliente` em endereços deve existir em clientes
 - Registre cada erro em um arquivo de log com: linha do arquivo, campo, valor, motivo da rejeição
 - Rejeite registros inválidos (não incluir no arquivo Parquet)
3. **Salve cada entidade como arquivos Parquet no bucket S3** na pasta `raw/`, mantendo os dados brutos
 - Adicione particionamento por data de processamento (`data_processamento=YYYY-MM-DD`)
 - Utilize compressão `snappy`

Parte 2 – Processamento e Camada Stage

4. Para cada entidade:
 - Considere que os dados representam **eventos de mudança**
 - **Para clientes**: mantenha apenas o último evento por `id_cliente`, com base no campo `data_evento`
 - **Para endereços**: mantenha apenas o último evento por `id_endereco`, com base no campo `data_evento` (não por `id_cliente`)

- **Utilize Delta Lake** para fazer o merge incremental dos dados nas pastas stage/clientes/ e stage/enderecos/ no S3
 - Implemente a lógica de **SCD Type 1** (Slowly Changing Dimension - sobrescrever)
5. **Requisitos técnicos desta etapa:**
- Use Spark para o processamento (não pandas)
 - Implemente tratamento de duplicatas
 - Adicione uma coluna data_atualizacao para rastreabilidade
 - Garanta idempotência do processo (executar múltiplas vezes deve produzir o mesmo resultado)
-

Parte 3 – Camada Analytics (20 pontos)

6. Realize um **join entre as entidades clientes e enderecos** com base no id_cliente:
- Considere apenas clientes com status “ativo”
 - Trate casos de clientes sem endereço cadastrado (LEFT JOIN recomendado)
 - Mantenha todas as combinações válidas: um cliente pode ter múltiplos endereços (id_endereco diferentes)
 - Adicione coluna calculada:
 - idade: calculada a partir da data de nascimento
7. **Salve o resultado** como arquivos Parquet na pasta analytics/clientes/ no S3:
- Otimize para consultas analíticas (considere ordenação e coalescing)
-

Parte 4 – Catálogo de Dados e Governança

8. **Crie um crawler no AWS Glue** programaticamente (usando boto3) que:
- **Nome do Crawler:** {nome}_{sobrenome}_crawler
 - **Database:** Use o banco de dados criado ({nome}_{sobrenome})
 - **S3 Target:** Aponte para a pasta s3://bkt-dev1-data-avaliacoes/{nome}_{sobrenome}/analytics/clientes/
 - **IAM Role:** {nome}_{sobrenome}_glue_crawler_role (procure o ARN completo no AWS Console ou construa usando seu account_id)

Nota: O banco de dados e a role já foram criados. Você apenas precisa criar e executar o crawler programaticamente usando boto3.

Parte 5 – Qualidade de Código e Boas Práticas

10. **Implemente os seguintes aspectos de engenharia de software:**
- **Logging estruturado:** registre início, fim e erros de cada etapa
 - **Tratamento de exceções:** capture e trate erros apropriadamente

- **Configuração externa:** use arquivo de configuração ou variáveis de ambiente para parâmetros (bucket, paths, etc.)
 - **Docstrings:** documente funções e classes seguindo PEP 257
 - **Type hints:** utilize anotações de tipo em funções
 - **Modularização:** organize o código em funções/classes reutilizáveis
11. **Testes** (opcional, mas valorizado):
- Implemente ao menos 2 testes unitários para funções críticas
 - Use pytest ou unittest
-

Parte 6 – Validação Final com Athena

12. Execute uma query no Amazon Athena para validar os dados processados:

- Execute programaticamente (usando boto3) a seguinte query no banco de dados {nome}_{sobrenome}:

```
SELECT * FROM clientes
```

- **Salve o resultado em um arquivo CSV** (resultado_athena.csv)
 - Use boto3 para executar a query e recuperar os resultados
 - Converta os resultados para um arquivo CSV
- Documente o resultado para validação

Objetivo: Validar que todo o pipeline funcionou corretamente e que os dados estão acessíveis via Athena.

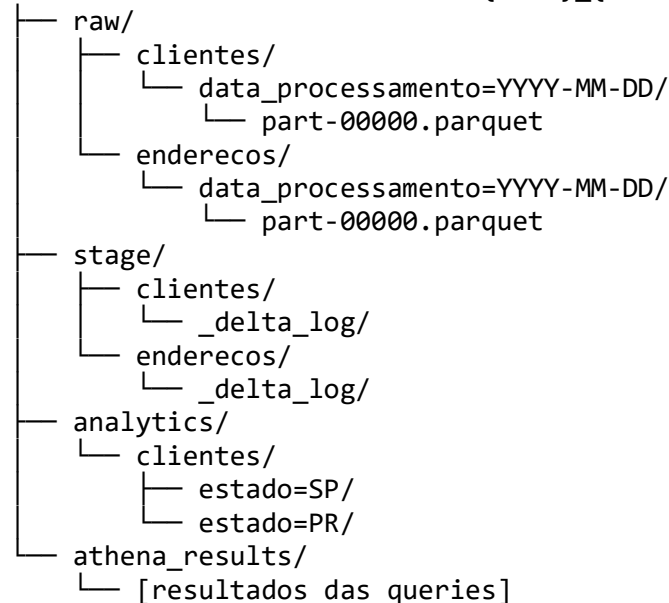
Requisitos Técnicos

- **Linguagem:** Python 3.8+
- **Bibliotecas obrigatórias:**
 - pyspark (para processamento com Spark)
 - delta-spark (para Delta Lake)
 - boto3 (para interação com AWS)
 - openpyxl ou pandas (para leitura do Excel)
- **Bibliotecas sugeridas:**
 - python-dotenv (para variáveis de ambiente)
 - pytest (para testes)
 - loguru ou logging (para logs)
- **Infraestrutura:**
 - Amazon S3 (armazenamento)
 - AWS Glue (catálogo)
 - Amazon Athena (consultas)
- **Engine de processamento:** Apache Spark 3.x com Delta Lake

Estrutura Esperada no S3

A estrutura será criada dentro de uma pasta com seu nome (em minúsculas com _):

s3://bkt-dev1-data-avaliacoes/{nome}_{sobrenome}/



Importante: Os particionamentos (como data_processamento=YYYY-MM-DD e estado=SP) devem ser construídos **dinamicamente** no seu código conforme você processa os dados.

Entregáveis

1. **Código fonte** organizado em módulos:
 - pipeline.py ou estrutura modular com múltiplos arquivos
 - config.py ou .env com configurações
 - requirements.txt com dependências
2. **Documentação** (README.md) contendo:
 - Instruções de setup do ambiente
 - Como executar o pipeline
 - Dependências e pré-requisitos
 - Decisões arquiteturais tomadas
 - Variáveis de ambiente necessárias:

S3_BUCKET=bkt-dev1-data-avaliacoes
AWS_REGION=sa-east-1

```
AWS_ACCESS_KEY_ID=<credencial_fornecida>  
AWS_SECRET_ACCESS_KEY=<credencial_fornecida>
```

3. **Scripts de infraestrutura** (programa seu próprio):
 - Script Python com boto3 para criação do crawler Glue
 - Referência de como usar a role fornecida pela infraestrutura
 4. **Testes** (se implementados):
 - Arquivos de teste (test_*.py)
 - Instruções de como executar os testes
 5. **Resultado da Validação Athena:**
 - Arquivo CSV com resultado da query (resultado_athena.csv)
 - Documentação da query executada
 6. **Logs de execução:**
 - Exemplo de log de uma execução bem-sucedida
-

Observações Importantes

- Você pode usar LocalStack ou configuração local de S3 para desenvolvimento
- Se não conseguir usar Delta Lake, documente a limitação e use Parquet com particionamento
- Priorize código limpo e bem estruturado sobre features complexas

Sobre Infraestrutura e Configuração:

- Os caminhos de pasta (raw/, stage/, analytics/), bucket, database e role do crawler já foram criados
 - **O crawler será criado por você** programaticamente como parte da Parte 4
 - Use arquivo de configuração (.env ou config.py) para armazenar esses valores centralizados (S3_BUCKET, AWS_REGION, etc.)
 - Construa os **particionamentos** (ex: data_processamento=YYYY-MM-DD, estado=SP) **dinamicamente** no seu código conforme processa os dados
-

Entrega

Envie o código fonte compactado (.zip ou link para repositório Git)

Boa sorte! 🚀